

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems. (BL3, BL6)*

Activity Tool : Analytical Problem Solving (High)

Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	5
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	5
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	5
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	5
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	5
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	5
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	5
8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity	BL3 – Apply	5

	constraint enforcement for a given scenario.		
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	5
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	5

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

ANSWERS

1. Apply relational algebra operations (σ , π , $\cup$, $-$, $\times$, $\bowtie$) on the given Employee and Department relations to retrieve required records.

Relational Algebra Operations on Employee and Department

Given Relations

Employee(Emp_ID, Emp_Name, Salary, Dept_ID)

Emp_ID	Emp_Name	Salary	Dept_ID
E01	Ravi	40000	D01
E02	Priya	50000	D02
E03	Arun	35000	D01
E04	Anu	60000	D03

Department(Dept_ID, Dept_Name)

Dept_ID	Dept_Name
D01	HR
D02	IT
D03	Finance

Operations

1. Selection (σ) – Select employees whose salary is greater than 40000.

$\sigma_{\text{Salary} > 40000}$ (Employee)

Result: Priya, Anu

2. Projection (π) – Display only employee names.

$\pi_{\text{Emp_Name}}$ (Employee)

Result: Ravi, Priya, Arun, Anu

3. Union ($\cup$) – Combine department IDs from Employee and Department.

$\pi_{\text{Dept_ID}}$ (Employee) $\cup$ $\pi_{\text{Dept_ID}}$ (Department)

Result: D01, D02, D03

4. Difference ($-$) – Find departments that have no employees.

$\pi_{\text{Dept_ID}}$ (Department) $-$ $\pi_{\text{Dept_ID}}$ (Employee)

Result: Empty set ($\emptyset$)

5. Cartesian Product ($\times$) – Combine every Employee with every Department.

Employee $\times$ Department

Result: $4 \times 3 = 12$ tuples

6. Join ($\bowtie$) – Join Employee and Department using Dept_ID.

Employee $\bowtie$ Employee.Dept_ID = Department.Dept_ID Department

Result: Employees are matched with their respective department names.

Conclusion

Relational algebra operations σ , π , $\cup$, $-$, $\times$, and $\bowtie$ are used to **select, display, combine, compare, pair, and join** data from relations.

2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.

SQL DDL – STUDENT and DEPARTMENT

Aim: To create STUDENT and DEPARTMENT tables using appropriate integrity constraints.

```
CREATE TABLE DEPARTMENT (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50) NOT NULL UNIQUE
);
```

```
CREATE TABLE STUDENT (
    SID INT PRIMARY KEY,
    Name VARCHAR(50) NOT NULL,
    DOB DATE NOT NULL,
    DeptID INT,
    FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
);
```

Constraints Used

- **PRIMARY KEY** – uniquely identifies each record.
- **NOT NULL** – ensures required fields contain values.
- **UNIQUE** – prevents duplicate department names.
- **FOREIGN KEY** – connects STUDENT.DeptID with DEPARTMENT.DeptID.

Result: The STUDENT and DEPARTMENT tables are successfully created with appropriate constraints.

3. Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.

SQL Aggregate Functions – Sales Dataset

Assume the table:

Sales(SaleID, Product, Amount, Quantity)

1. COUNT – Number of sales for each product

```
SELECT Product, COUNT(*) AS Total_Sales
```

```
FROM Sales
```

```
GROUP BY Product;
```

2. SUM – Total sales amount for each product

```
SELECT Product, SUM(Amount) AS Total_Amount
```

```
FROM Sales
```

```
GROUP BY Product;
```

3. AVG – Average sales amount

```
SELECT Product, AVG(Amount) AS Average_Amount
```

```
FROM Sales
```

```
GROUP BY Product;
```

4. MAX – Maximum sales amount

```
SELECT Product, MAX(Amount) AS Maximum_Amount
```

```
FROM Sales
```

```
GROUP BY Product;
```

5. MIN – Minimum sales amount

```
SELECT Product, MIN(Amount) AS Minimum_Amount  
FROM Sales  
GROUP BY Product;
```

6. HAVING – Products with total sales above 50,000

```
SELECT Product, SUM(Amount) AS Total_Amount  
FROM Sales  
GROUP BY Product  
HAVING SUM(Amount) > 50000;
```

- **COUNT()** → counts records
- **SUM()** → calculates total
- **AVG()** → calculates average
- **MAX()** → finds highest value
- **MIN()** → finds lowest value
- **GROUP BY** → groups records based on a column
- **HAVING** → filters grouped results

4 Write a correlated sub-query to find employees earning more than the average salary of their own department.

Assume the table:

Employee(EmpID, EmpName, Salary, DeptID)

SQL Query

```
SELECT E.EmpID, E.EmpName, E.Salary, E.DeptID  
FROM Employee E  
WHERE E.Salary > (  
    SELECT AVG(E2.Salary)  
    FROM Employee E2  
    WHERE E2.DeptID = E.DeptID  
);
```

- The **outer query** selects employees.
- The **inner query** calculates the average salary of the employee's department.
- E2.DeptID = E.DeptID makes it a **correlated sub-query** because the inner query depends on the current row of the outer query.

Result: Employees whose salary is **greater than their own department's average salary** are displayed.

5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables

JOIN Operations – Employee and Project

Assume the tables:

- Employee(EmpID, EmpName, ProjectID)
- Project(ProjectID, ProjectName)

1. INNER JOIN

Returns only matching records from both tables.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
INNER JOIN Project P
ON E.ProjectID = P.ProjectID;
```

2. LEFT OUTER JOIN

Returns **all employees** and matching projects.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
LEFT JOIN Project P
ON E.ProjectID = P.ProjectID;
```

3. RIGHT OUTER JOIN

Returns **all projects** and matching employees.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
RIGHT JOIN Project P
```


ON E.ProjectID = P.ProjectID;

4. FULL OUTER JOIN

Returns **all employees and all projects**. MySQL does not directly support FULL OUTER JOIN, so use UNION.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
LEFT JOIN Project P
ON E.ProjectID = P.ProjectID
```

UNION

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
RIGHT JOIN Project P
ON E.ProjectID = P.ProjectID;
```

5. CROSS JOIN

Returns **every possible combination** of employees and projects.

```
SELECT E.EmpName, P.ProjectName
FROM Employee E
CROSS JOIN Project P;
```

Summary

JOIN	Purpose
INNER JOIN	Matching records only
LEFT JOIN	All employees + matching projects
RIGHT JOIN	All projects + matching employees
FULL OUTER JOIN	All records from both tables
CROSS JOIN	Every possible combination

Result: All major JOIN operations are applied successfully to the Employee and Project tables.

5. Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.

Create View — DeptSalaryView

Assume the tables:

- Department(DeptID, DeptName)
- Employee(EmpID, EmpName, Salary, DeptID)

1. Create the View

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
       COUNT(E.EmpID) AS TotalEmployees,
       AVG(E.Salary) AS AverageSalary
FROM Department D
LEFT JOIN Employee E
ON D.DeptID = E.DeptID
GROUP BY D.DeptID, D.DeptName;
```

2. Query the View

```
SELECT * FROM DeptSalaryView;
```

- COUNT() → finds the total number of employees.
- AVG() → finds the average salary.
- GROUP BY → groups employees department-wise.
- LEFT JOIN → also displays departments having no employees.
- VIEW → creates a virtual table based on the query.

Result: DeptSalaryView displays **Department Name, Total Employees, and Average Salary** for each department.

7. Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.

Relational Algebra – Students Enrolled in All Courses

Assume the relations:

- **STUDENT**(SID, Name)
- **ENROLL**(SID, CID)
- **COURSE**(CID, CourseName)

Expression

First, find students enrolled in **all courses** using **Division** ($\div$):

$$R = \pi_{SID, CID} (ENROLL) \div \pi_{CID} (COURSE)$$

Then find the student names:

$$\pi_{Name} (STUDENT \bowtie R)$$

Explanation

- π (**Projection**) selects required attributes.
- $\div$ (**Division**) finds students enrolled in **every course**.
- $\bowtie$ (**Join**) connects the result with STUDENT to obtain names.

Final Answer:

$$\pi_{Name} (STUDENT \bowtie (\pi_{SID, CID} (ENROLL) \div \pi_{CID} (COURSE)))$$

8. Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.

SQL DML – INSERT, UPDATE, DELETE

Assume the tables:

- DEPARTMENT(DeptID, DeptName)
- STUDENT(SID, Name, DOB, DeptID)

1. INSERT

```
INSERT INTO DEPARTMENT VALUES (10, 'Computer Science');
INSERT INTO STUDENT VALUES (101, 'Ravi', '2005-06-10', 10);
```

2. UPDATE

```
UPDATE STUDENT SET DeptID = 10 WHERE SID = 101;
```

3. DELETE

```
DELETE FROM STUDENT WHERE SID = 101;
```

Integrity Constraint Enforcement

- SID and DeptID are **Primary Keys** and cannot be duplicated.
- Name and DOB are **NOT NULL**.
- STUDENT.DeptID is a **Foreign Key** referencing DEPARTMENT.DeptID.

- Therefore, a student cannot be inserted or updated with a DeptID that does not exist in DEPARTMENT.

Result: The INSERT, UPDATE, and DELETE operations maintain database integrity by enforcing the defined constraints.

9. Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.

Assume:

- EMPLOYEE(EmpID, EmpName, Salary, DeptID)
- DEPARTMENT(DeptID, DeptName)

TRC Expression

$\{e \mid \text{Employee}(e) \wedge e.\text{Salary} > 50000 \wedge \exists d(\text{Department}(d) \wedge e.\text{DeptID} = d.\text{DeptID} \wedge d.\text{DeptName} = \text{'IT'})\}$

Explanation

- Employee(e) → selects employee records.
- e.Salary > 50000 → selects employees earning more than 50000.
- Department(d) → checks the department relation.
- e.DeptID = d.DeptID → connects employee with department.
- d.DeptName = 'IT' → selects only IT department employees.

Result: Employees working in IT with a salary **greater than 50000** are retrieved.

10, Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.

Airline Reservation System

1. Relational Schema

PASSENGER

Passenger(PassengerID, Name, Phone, Email)

FLIGHT

Flight(FlightID, Source, Destination, FlightDate, Seats)

RESERVATION

Reservation(ReservationID, PassengerID, FlightID, BookingDate, SeatNo)

Relationships

- PassengerID → Foreign Key in Reservation
 - FlightID → Foreign Key in Reservation
 - One passenger can make many reservations.
 - One flight can have many reservations.
-

2. Five SQL Queries

Query 1 – Join

Display passenger names and their flight details.

```
SELECT P.Name, F.Source, F.Destination FROM Passenger P JOIN Reservation R ON P.PassengerID = R.PassengerID JOIN Flight F ON R.FlightID = F.FlightID;
```

Query 2 – Join with Condition

Find passengers travelling to Chennai.

```
SELECT P.Name, F.FlightID FROM Passenger P JOIN Reservation R ON P.PassengerID = R.PassengerID JOIN Flight F ON R.FlightID = F.FlightID WHERE F.Destination = 'Chennai';
```

Query 3 – Sub-query

Find flights having more seats than the average number of seats.

```
SELECT * FROM Flight WHERE Seats > (  
    SELECT AVG(Seats)  
    FROM Flight  
);
```

Query 4 – Create a View

Create a view showing passenger and flight details.

```
CREATE VIEW ReservationView AS SELECT P.Name, F.Source, F.Destination, R.SeatNo FROM  
Passenger P JOIN Reservation R ON P.PassengerID = R.PassengerID JOIN Flight F ON R.FlightID =  
F.FlightID;
```

Query 5 – Query the View

```
SELECT * FROM ReservationView;
```

Conclusion

The Airline Reservation System uses **relational tables, primary/foreign keys, joins, sub-queries, and views** to efficiently manage passenger, flight, and reservation information.